- If no bytes remain on either, the strings are the same (return `0`).
- If bytes remain on one but not the other, the other is smaller (return `-1` or `1`).

This will hopefully make more sense if we switch to hex escapes in our strings. The letter M is (in UTF-8 and ASCII) a single-byte character *0x4d*; the digits *0–9* are also single bytes in the range *0x30–0x39*.

```
$m4  = "\x4d\x34";
$m1  = "\x4d\x31";
$m11 = "\x4d\x31\x31";
$m18 = "\x4d\x31\x38";

var_dump(strcmp($m4, $m11));
var_dump(strcmp($m4, $m18));
var_dump(strcmp($m11, $m18));
var_dump(strcmp($m1, $m11));

Output:
int(3)
int(3)
int(-7)
int(-1)
```

All strings begin with byte *0x4d* and will begin to vary on the second or third byte. Thus, `$m4` is greater than `$m1` by three because `0x34-0x31 = 3`, and `$m11` is seven less than `$m18` because `0x31-0x38 = -7`. The comparison of `$m1` and `$m11` returns `-1` because of the difference in byte length. Once you understand that strings are just streams of bytes (that can also be treated as arrays), the binary comparison becomes obvious when looking at the results.

The `strcasecmp()`, `strnatcmp`, and `strnatcasecmp()` functions introduce a little more logic to the comparison algorithm. The functions with `case` in their names ignore case, which means that `"A"` and `"a"` are equivalent, as are `"B"` and `"b"`, and so on; therefore, `strcasecmp("AbC", "aBc");` would return `0`. The `nat` functions also introduce natural sorting, in that numerical strings will have entire numbers considered, not just a digit-by-digit comparison:

```
var_dump(strnatcmp("M4", "M18"));
var_dump(strnatcmp("M40", "M18"));
```